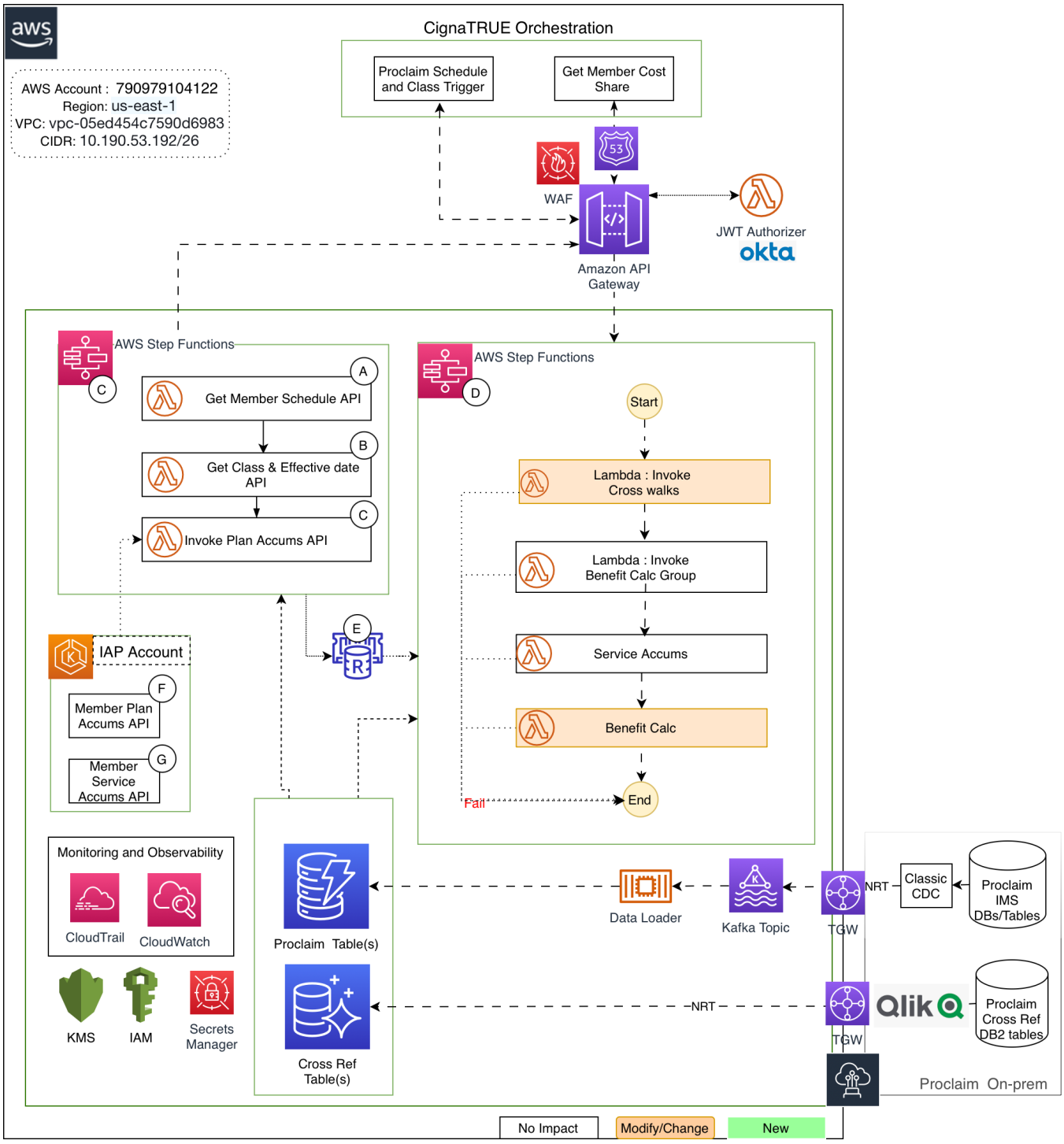
CIGNA True

Design Diagram:

Copied from Ravi Architecture Diagram:



4. Application/Component Architectural View - 07792 CignaTRUE

Cigna TRUE - Logical Architecture - MVP1

Data contract:

CIGNA True Data Contract:

Compute logic:

Cross walk Lambda: MVP1

- Accept the modifiers from the input request.
- Match the modifier while doing the service type lookup.

Get Plan/Provider PPO information: MVP1

All the below functionality implemented for AEOB and not anticipating major changes in the below path.

- CET does the contract selection but it won't get the plan information.
- Need to implement logic to pull all the plan related information as like claim payment.
- Program flow:
    - Read the earlier start date from serviceFromDate and hold it.
    - Compare the earlier start date with input networkEffectiveDate and networkCancelDate, and hold dates(holdNetworkEffectiveDate , holdNetworkCancelDate) if claim service dates within the network effective and cancel date.
    - Read Provider PPO(New Dynamo DB) for the input claim engine network ID(claimEngineNetworkId) and the tax indicator 'P'.
    - Return the error if PPO not found in the DB.
    - Return error if there are multiple networks
    - Hold the RNC code(524P-GCPRV-PPO-RNC-CODE) and Effective and Cancel dates from Provider PPO table.
    - Check for midlevel practitioner, networkReimbersmentLineType should be 'L' and set the MLP find indicator(mlpFound) and earlier start data must be networkReimbersmentLineEffectiveDate and networkReimbersmentLineCancelDate range.
    - Check for PCP, networkReimbersmentLineType should be 'R' and set the PCP find indicator and earlier start data must be networkReimbersmentLineEffectiveDate and networkReimbersmentLineCancelDate range.
    - Check for CCN, networkReimbersmentLineType should be 'N' and set the CCN find indicator and earlier start data must be networkReimbersmentLineEffectiveDate and networkReimbersmentLineCancelDate range.
    - Check for MMM, networkReimbersmentLineType should be 'M' and set the MMM find indicator and earlier start data must be networkReimbersmentLineEffectiveDate and networkReimbersmentLineCancelDate range.
    - Read contract selection methodology table (new table need to provision in cloud), and see whether the input retunred PPO exist in that table or not, need to match the exact and wild PPO's. Set the indicator appropriatly(holdNetworkFound).
    - Lookup into ALLPPO table for the exact or wild PPO when no match found in contract selection methodology table and set the indicator appropriately(holdNetworkFound).
    - Read GCSOCNPO table for all the entries as per the hierarchy.
    - Match the exact PPO or wild PPO as per the hierarchy.
    - Any input PPO can match with the GCSOCNPO wild PPO when the holdNetworkFound is 'Y'.
    - Loops through SOCNPO table entries to find a match based on PPO ID, effective/termination dates, and hierarchy type.
    - If a match is found, moves the matched segment to a hold area for further processing.
    - Return the error if CLASS-TIER-IND is 'B' and SOCNPO BEN-NTWK indicator NOT 'Y.
    - Return the error if CLASS-TIER-IND is 'T' or 'W' and SOCNPO CDB-NTWK indicator NOT 'Y.
    - If a PPO ID match is found, then load all the plan information.
        - SOCNPO-TIER-LVL-MC
        - WS-HOLD-SOCNPO-CCN-IND
        - SOCNPO-PRV-TYP
    - Set the providerNetworkIndicator:
        - Default to 'N'
        - Input provider type is +57 +68 +69 +73 and SOCNPO-PRV-TYP is 'PC' then set it to 'P'.
        - Input provider type is +07 +57 +68 +69 +73 and SOCNPO-PRV-TYP is 'PG' then set it to 'P'.
        - SOCNPO-PRV-TYP is 'RD' and PCN-PLAN-PARTIC value is 'Q' or 'R' or 'X' or 'Z' or 'C' or 'B' and mlpFound is 'Y' then set to 'P'.
        - SOCNPO-PRV-TYP is 'PC' or 'PG' and mlpFound is 'Y' then set to 'P'.
        - Otherwise set to 'O'.
    - Set the the network as OON if no PPO ID match found.

Get Plan Benefits: MVP1

- Enhance the logic to include the modifiers in the benefits lookup.
- Match the request modifier1,2,3 and 4 with the GCCVGSEL database Modifier MODIFIER-1 when it is greater than blank.
- Match the request modifiers modifier1,2,3 and 4 with the pclm_benadj.gcslscrt database Modifier mod_code when it is greater than blank.

Apply Pricing(SANP pricing response translator) and CXT rules:

CXT Rules: MVP1

- Accept the below CXT data from request JSON.

```
"caseAction": " ",
"caseCertainty": " ",
"caseJustificationId": " ",
"caseRule": " ",
"caseAuditType": " ",
"cslLineAction": " ",
"cslJustificationId": " ",
"cslRvu": " ",
"cslRuleType": " ",
"cslAdjustedcode": " ",
"claimClaimId": " ",
"claimlRvu": " "
```


- Read the new Dynamo table CXT-RNC-XWALK (GQDN.CXT_RNC_XWALK, new table need to load in Dynamo) for the concatenated key ruleAcrnym_pathCode_typeCode_explanCode.
- And below are the fields need to map to manufacture the concatenated key, this table will have hardly 2K records, we can read whole table and hold it in work area instead making multiple IO calls.
    - cslAdjustedcode, it need to translate to upper case before querying and is nothing but explanCode.
    - caseRule it is nothing but ruleAcrnym.
    - cslRuleType it is nothing but pathCode.
    - auditType, this field need to derive as like below logic and it will be mapped to typeCode
        - Scan for caseAction
            - Set the auditType value to C-C when caseAction is EXPAND_QTY.
            - Set the auditType value to R-C when caseAction is MODIFY.
            - For the caseAction DENY:
                - Set the auditType value to C-H when caseAction is DENY and clmHistoryFlag is 'Y', it is out of scope for MVP1 as history not sharing with CXT.
                - Set the auditType value to C-N when caseAction is DENY and cslLineAction is ADD for any instance for that line.
                - Set the auditType value to C-C when caseAction is DENY and cslLineAction is SUPPORT for any instance for that line.
                - Otherwise set the auditType value to D-C
            - For the caseAction TRANSFER:
                - Set the auditType value to H-N when caseAction isTRANSFER, cslLineAction is ADD and clmHistoryFlag is 'Y' and history cslLineAction is DENY , it is out of scope for MVP1 as history not sharing with CXT.
                - Set the auditType value to C-N when caseAction is TRANSFER and cslLineAction is ADD and cslLineAction is 'DENY for any instance for that line.
                - Return the error then the caseAction is TRANSFER and cslLineAction is ADD and above two rules not satisfied.
                - Set the auditType value to C-H when caseAction is TRANSFER and history cslLineAction is SUPPORT but out of scope for MVP1.
                - Set the auditType value to C-C when caseAction is TRANSFER and current cslLineAction is SUPPORT.
                - Otherwise return the error.
            - For the caseAction PAYPERCENT:
                - Set the auditType value to P-H when caseAction is PAYPERCENT and history cslLineAction is SUPPORT, out of scope for MVP1.
                - Set the auditType value to P-C when caseAction is PAYPERCENT and current cslLineAction is SUPPORT.
                - Otherwise set the auditType value to P-P.

Pricing Rules: ->MVP2

- Check for Pricing Status code for 1005 proceed otherwise return error.
- Derive the price type as like below.
  IF pricerType is 'I' then 'APC' pricing
  IF pricerType is '64' then 'ASC' pricing
  IF pricerType is 'S' the set 'DRG' then DRG or case rate pricing
  IF based on code is -F or -N and pdUsage is 'S' then 'SOS' pricing.
- Logic to set the case rate indicator.
    - check for line-num 8888 or 9999 and usage is A or SA then set case rate indicator to true.
- Logic to set the stop loss indicator.
    - check for line-num 9999 and usage is A or SA then set stop loss indicator to true.
- Derive the pricing attribute as like below
    - IF case rate indicator is 'Y' and used-drg > spaces then DRG pricing and schedule attribute would be DRGOP
    - IF case rate indicator is 'Y' and used-drg <= spaces then Case rate pricing schedule attribute would be OTROP
    - IF Pricer type is 'I' then schedule attribute would be ASCOP
    - IF Pricer type is '64' then schedule attribute would be APCOP
    - IF Pricer type is 'S' then DRGOP' Do we need to create DRG line as like claim?
    - IF based on code is -F or -N then schedule attribute would be SOSOP
    - IF pricing attribute is spaces then look for line level group charges > charges then set pricing attribute to OTROP otherwise follow the line level pricing.
- Set the line level pricing for ASCOP,APCOP,SOSOP and NON-PAR.
- Set the group pricing for DRGOP and OTROP.
- Return the error code 7011 when total allowed amount = 0 for the entire claim & none of the rate type fields contain ‘Denied’
- Apply lessor of language when the stop-loss indicator is 'Y' and schedule attribute ASCOP or APCOP.
    - Set Allowed = billed charges.
    - Not applying the plan discounts, need to check.

        - And return the process with successful status code.
- Pricing logic for ASCOP,APCOP,SOSOP:

    i. Scan through pricing details for 9999 line and based on code is 'S'. If found, process as LOL (Lesser of Language) and populate billed charges or plan covered expenses, whichever is less.
    ii. Read through Facility line items array
    iii. Use Allowed Charges as Covered amount for service line
    iv. Not covered amount for service line = Billed – Covered amount. If pricing allowed is less than plan covered then the plan RNC line is as described above and then an additional SANP RNC line is needed for the additional savings. This is for Proclaim only. PPN will not have a not covered amount.
    v. If Covered amount for service line > Bill charges then move bill charges to Covered amount and accumulate the difference in new claim level excess amount field. This excess amount will be used in the process of ANC line creation or +line(Upon LDTP scope).

- Pricing logic for DRGOP.
    - If there is a ‘8888’ line with usage =A/SA & UsedDRG <> spaces
        - Read through all the Facility line items array
        - If IsExcludedItem = "true" then use Allowed Charges from that line as Covered amount for service line. Not covered amount for service line = Billed – Covered amount. If pricing allowed is less than plan covered then the plan RNC line is as described above and then an additional pricing RNC line is needed for the additional savings.
        - If IsExcludedItem = "false” this means line has grouped amount/case rate; get Allowed Charges, bill charges from all service line with IsExcludedItem = "false” and sum-up.
        - When there is a IsExcludedItem = "false” on any one Facility line items then read through Pricing Details to see if there is a ‘8888’ line exists with usage ‘A’ or ‘SA’.
        - If there is a Pricing detail line ‘8888’ with usage = A or SA and UsedDRG is populated then
            - Create new DRG line.
            - If total group allowed charges (DRG amount) > total group bill charges (refer rule#1), then do not prorate, instead on DRG line, move total group allowed charges to Bill charges. Not covered amount will be calculated in pass-2. Otherwise perform below
            - Populate ‘DRG’ line and total group bill charges, total group allowed charges.
            - Prorate above group allowed amount to all service lines with IsExcludedItem = "false".
                - If IsExcludedItem = "false"
                    - Get corresponding billed charges/plan covered expense limit (from SANP IO) (refer rule#1), for current solution we are assuming plan covering all the allowed charges for the covered services.
                    - Covered amount for service line = (Bill charges/Total Group bill Charges) * Total Allowed Charges (refer rule#1)
                    - Not covered amount for service line = Bill charges – Covered amount.
                    - If there are any service lines that are Line level priced (along with DRG priced lines), and if the Pricing Allowed amount is greater the billed charges then set 7001 error code and move
                    - Covered amount on all the excluded lines from the DRG pricing should be considered to update the charged amount on DRG line along with included lines.
    - If there is a no ‘8888’ line with usage =A/SA & UsedDRG <> spaces
        - When there is no 8888 line exists with usage A/SA in pricing details, then don't create DRG line and follow Case rates /PDM logic.

- Pricing logic for Case Rate(OTROP):
    - Read through Facility line items array.
    - If IsExcludedItem = "true" then use Allowed Charges from that line as Covered amount for service line. Not covered amount for service line = Billed – Covered amount. If SANP allowed is less than plan covered then the plan RNC line is as described above and then an additional SANP RNC line is needed for the additional savings.
    - If IsExcludedItem = "false” this means line has grouped amount/case rate; get Allowed Charges and bill charges from all service line with IsExcludedItem = "false” and sum-up.
    - Prorate resultant grouped amount to all service lines with IsExcludedItem = "false".
    - If IsExcludedItem = "false"
        - Get corresponding billed charges/plan covered expense limit (from SANP IO) (refer rule#1), assuming plan covered expenses always 100%.
        - Covered amount for service line = (Bill charges/Total Group bill Charges) * Total Allowed Charges (refer rule#1)
        - If Covered amount for service line > Bill charges then move bill charges to Covered amount and accumulate the difference in new claim level excess amount field. This excess amount will be used in the process of ANC line creation or + line creation(upon LDTP scope).
- Generic Rules and Exceptions:
    - While prorating group charges take either Bill charges or plan covered expense limit whichever is lesser from request. Also take either Total Group Charges or total plan covered expense limit of all service lines whichever is lessor. **While prorating, if the Plan covered limit is all 9’s, consider the Bill charge.**
    - If total allowed amount = 0 for the entire claim & none of the rate type fields contain ‘Denied’ and set 7011 error code and ‘No pricing received - refer to SANP rate sheet for manual calculation’ to error message (Claim will be routed to claim processor for manual intervention)
    - For 8888 and 9999 lines, If we have pricing details with usage SA & A then consider only A. If there is no 'A' then take 'SA'.
    - Line-Denied-015
        - APC and ASC:
        - Apply RNC with zero allowed (1867) using below RNC apply process. Request will not be errored.
- Rules to apply RNC:
    - Append the new RNCs to existing RNCs (that are applied before pricing logic), need to create next order if there is RNC present on current order.

    - Do not apply the same RNC if one is already present in orders for the same service line
    - Pricing Details
        - Read Pricing details array for a given service line using pointer pdPtr available in facility line item array for better performance.

- Rate Type Field will house the reasons that a service line may be denied or partially covered.
- Based on Code will house the status indicators which state the process for which the claim was priced.
- Rate type fields takes precedency over Based on while applying RNC
- If no Rate Type or Based on Code is available or the value is ‘not mapped’, utilize the default code based upon the PPO information (normal RNC application), PPO RNC will come from network lambda and now it is combining with XWALK.
    - For Proclaim DRG priced claim- Call RNCDBM to get PPO RNC with call type as ‘GCPDMRNC’ and populate PPO id, we may need additional table in cloud, **analysis in progress.**
        - If RNC is not found move +263
        - Access the amount required indicator from the XWalk when available, otherwise access RNCDBM for the default PPO RNC or +0263 RNC to check the amount required indicator
    - For all other claims- Use PPO RNC

- Hierarchy in which RNCs to be mapped
    - OCE edits (eg:- LN-Denied-010, CLM-Denied-010); From Rate type field
    - Payment Status indicators- PSI (eg:- SI-T, SI-N); From Based on field
    - APC/ASC return codes (eg:- 42, P10); From Rate type field
- How to map OCE edit/SI/Return codes to Proclaim/PPN RNC/Error
    - SANP RNC Cross walk table- GQDN.SANP_RNC_XWALK

CD_TYPE X(10)	CD_VALUE X(5)	PCLM_RNC SMALLINT	PPN_RNC SMALLINT	ERROR_CD SMALINT	LAST_UPDT TIMESTAMP	LAST_UPDT_USER X(08)	RLSE_NOTE X(16)
OCE	005	1813	??		CURRENT TIMESTAMP	INITLOAD	FFS GAPS
OCE	DFLT	1855	??		CURRENT TIMESTAMP	INITLOAD	FFS GAPS
SI	S	1859	??		CURRENT TIMESTAMP	INITLOAD	FFS GAPS
SI	D5	1882	??		CURRENT TIMESTAMP	INITLOAD	FFS GAPS
OCE	001			1454	CURRENT TIMESTAMP	INITLOAD	FFS GAPS

- Rate type field will be parsed into 6 fields & Based on code field will be parsed into 3 fields by SI.
- Read through all 6 Rate type fields first,
    - If field value has 3 digit numeric then, call SANP-RNC-Xwalk table with CD-TYPE = ‘OCE’ and CD-VALUE = 3 digit numeric value from that field and get corresponding Proclaim/PPN RNC & Error code
    - Else if value is ‘CLM-Denied’ & no other rate type field has 3 digit numeric value then
        - Call SANP-RNC-Xwalk table with CD-TYPE = ‘OCE’ and CD-VALUE = DFLT and get corresponding Proclaim /PPN RNC & Error code
    - Else if call SANP-RNC-Xwalk table with CD-TYPE = ‘PRICERCD’ and CD-VALUE = first 3 bytes from rate type field an d get corresponding Proclaim/PPN RNC & Error code
- Read through all 3 Based on code fields
    - If field value contains ‘SI-‘ then, call RNC-Xwalk table with CD-TYPE = ‘SI’ and CD-VALUE = 2 digits after ‘SI-‘ from that field and get corresponding RNC & Error code
- If RNC-Xwalk table results is a valid RNC code (> 0) then use RNC code; if it returns valid error code (>0 i.e. claim need to be routed to CP) then return error code as 7004 and error message as ‘Send claim to manual processor’. Move error code from Xwalk table to additional error area in the response.
- If both RNC code and error code are blank or no records found for the input from Xwalk table then use default PPO RNC.
- Proceed to the next service line and apply RNC/Error code. Do not stop process until alffcl service lines are processed (even when DET error occurs)
- The RNC amount required indicator is being added to the crosswalk table. This indicator needs to be used when determining what RNC to use if there is no not covered amount.

Accum Calculation: MVP2

- Business Objective:

    When a member receives a medical service, someone must decide **how much the member pays** and **how much the health plan (Cigna) pays**. This service is the engine that makes that split.
    For a given member, a given date of service, and one or more providers, it calculates the member's **cost share** by applying the plan's benefit rules in this order:

    1. **Deductible** — the amount the member must pay before the plan starts paying.
    2. **Copay** — a fixed charge for the service.
    3. **Coinsurance** — the member's percentage share after the deductible.
    4. **Maximum (Max)** — caps on what the plan will pay (by dollar amount or by number of occurrences / visits).
    5. **Not-Covered amount** — anything the plan will not pay at all.
    6. **New York State Surcharge (NY SSC)** — an extra state-mandated charge added to certain New York services.

    The final result tells the downstream claims system the member's deductible, copay, coinsurance, out-of-pocket (OOP) total, the plan's share, and the not-covered amount — broken down per provider and per benefit "tier".

- **Request:**

The service receives one JSON request per transaction. Key inputs:
Member-level information (`costshare_mem_output` + common fields)
- Employee SSN, subscriber & patient IDs
- Patient number, patient date of birth
- **SOC number** (Schedule of Coverage), **SOC class**, **class effective date** —these identify which benefit plan applies
- Plan schedule code, patient relationship code
- Surcharge election indicators (SOC-level and class-level)
- Surcharge exclusion indicator (`ssc_excl_ind`)
- Operator location (`oploc`), SOC accounting type
- Request date (the date of service used for all calculations)
- Claim engine ID (e.g. `PCLM`)

Service-level information (`common`)
- Procedure code and type (e.g. CPT4 `87086`)
- Diagnosis code, revenue code, DRG code, facility/billing details

Provider-level information (`providers[]`, one or more)
- Contracted (priced) rate — the amount being adjudicated
- Provider TIN and TIN type (EIN vs SSN)
- **Select group** and **Calc group** — pointers into the plan definition that determine which benefits apply
- Network indicator (INN / OON), place of service
- Reason-denied code, span-date indicator, accumulation-occurrence indicator
- Provider TLC code (a 3-character classification)
- State surcharge code (e.g. `NY1`) and provider ZIP code
- A per-provider message `code` (`0200` = success)

Accumulators (prior usage) — `accums[]` at both plan and provider level.
These represent how much of each deductible / coinsurance / maximum the member has already used in the relevant period. Each entry has an ID, a value, a family/individual indicator, a value type (dollar or occurrence), and a date range.

Overall transaction status — `status_code` (HTTP-style) and a member-level `message.code`.

- 2.2 Reference data (looked up by the service)

Source	Store	What it provides
GCCVGDEF	DynamoDB	Coverage **Definition** — for a Calc Group, lists up to 5 benefit tiers and which deductible / coinsurance / maximum groups apply to each
GCCVGDCM	DynamoDB	Coverage **Detail** — the actual deductible (DED), coinsurance (COI), and maximum (MAX) group definitions: limits, percentages, periods, connectors
GCELGDAT	DynamoDB	Member **eligibility / salary** data (used for salary-based deductibles)
ZIP_INFO	RDS (Postgres)	Maps a ZIP code to a GME region (for NY surcharge)
STATE_SURCHARGE	RDS (Postgres)	NY surcharge percentages by election indicator / GME region
OPLOC_STATE_SURCHARGE	RDS (Postgres)	Operator-location-level surcharge election overrides

- 3. Processing Flow (step-by-step, in business language)

Stage A — Accept and screen the request

1. The request arrives either via AWS Lambda or an HTTP endpoint(`/proclaim/bencalc/v1/calculation_lambda_post`).
2. **If the overall transaction status is not "OK" (200)** stop; return an error response describing all providers as errored.
3. **If the member-level message code cannot be read as a number** stop; return a bad-request error response.
4. **If there are no providers in the request** return status **202** ("all providers bypassed").
5. **If the member message code is not `0200` (success) or `0201` (partial success)** stop; return an error response carrying that code.

Stage B — Separate good providers from bad providers

6. Split the providers into **success providers** (provider message code `0200`) and **error providers** (any other code).
7. **If there are no success providers** return an error response (nothing to calculate).
8. Only the success providers proceed to calculation. Error providers are added to the output untouched (with their original codes) at the end.

Stage C — Validate the input fields (member + each provider)

9. Validate required member fields and build the plan lookup key. The plan key(`GcCvgDEFKeyVal`) is constructed as: **5-digit SOC number + class effective date + line-of-business + 4-digit SOC class.**
10. For each provider, confirm the priced amount, select group, and calc group are present and valid. While looping, detect whether **any** provider has a New York surcharge (`NY` prefix) sets the "NY surcharge applies" flag.

11. **If any validation fails** stop; return an error response with the specific error code (see Decision Matrix and Error Handling).

Stage D — Fetch reference data (in parallel)

12. In parallel (for performance), the service retrieves:

- **Coverage Definitions (GCCVGDEF)** for every distinct Calc Group in the request (deduplicated so each Calc Group is fetched once).
- **Coverage Details (GCCVGDCM)** for the plan key.
- **Eligibility/salary data (GCELGDAT)** for the employee (skipped for "SAR" request sources).
- **Operator-location surcharge data (OPLOCSSC)** — only if NY surcharge applies and surcharge is not excluded.

13. **If Coverage Detail (GCCVGDCM) is missing** stop; member-level error `6029` ("CVGDEF/CVGDCM call failed").
14. **If Eligibility data is missing AND the claim engine is `PCLM`** stop; member-level error `6002`. (For non-PCLM sources, missing eligibility is tolerated.)
15. From eligibility, select the **first record whose effective date is on or before the request date** and capture the member's salary indicators and amounts (used later for salary-based benefits).

Stage E — Calculate cost share for each provider (in parallel)

For **each** success provider the engine performs the following:

16. **Pre-checks**

- If the provider has no Calc Group but has a reason-denied code provider error `6024` ("provider has no calc group").
- If the provider TLC code is shorter than 3 characters provider error `6043`.

17. **State surcharge** — if the provider has a surcharge code and a ZIP code, run the NY surcharge routine (see Stage F). The surcharge amount is added to the contracted rate to form the "held contracted rate" used in calculations. If no surcharge applies, the surcharge amount is zero.

18. **Locate the plan definition** for this provider's Calc Group. If it was not found earlier provider error `6029`. If the definition is empty provider error `6028` ("calc group not found in plan").
19. **Seed the working area** with the surcharge, contracted rate, type of service, patient details, dates, salary data, the prior accumulators, and the 3rd character of the TLC code (used for Medicare/injury checks).
20. **Walk the benefit tiers (up to 5)** — see "Tier calculation" below.
21. **Final per-provider math** once tiers are done:

- Total member accumulations = deductible + coinsurance + copay.
- **Plan (Cigna) share** = the total maximum-covered amount (rounded).
- **Not-covered amount** = |contracted rate + NY surcharge (member accumulations + plan share)|.
- **Customer cost share** = member accumulations + not-covered amount.
- If there is a not-covered amount and no reason code yet, assign the

plan's reason-denied code (or the tier-specific one).#### Tier calculation (repeated for each of up to 5 tiers)

Each Coverage Definition can have up to **5 tiered benefits**. For each tier, in order:

a. **Stop** if the tier's coverage code is 0 (no more benefits defined).
b. On the first tier, set the charged & covered amount to the held contracted rate.
c. **Medicare / accident-injury eligibility check** — if the tier specifies an accident-injury or Medicare-eligibility indicator, compare it to the member's situation (TLC code = "J" means injury; member is Medicare-eligible if they have reached age 65 by the date of service). If the indicators don't match the member, **skip this tier** with a note "member eligible for Medicare, no calculation".

d. **Maximum pre-check** — if the tier has a maximum group, check whether the member has **already met** that maximum (dollar or occurrence). If met, the tier is closed: set the not covered reason and stop calculating this tier. A secondary "basic maximum" group is checked the same way if present.

e. **Deductible & copay** (only if the maximum is not already met):

- Gather the deductible group and all connected deductible groups.
- Determine each group's limit based on its type (see Business Rule set on deductibles), the member's prior usage, and salary if applicable.
- Calculate the deductible and/or copay amount the member owes for this service.

f. **Remaining covered amount** = covered amount deductible copay.

g. **Coinsurance** (only if the maximum is not met) — determine the applicable coinsurance group, cutoff level, and payable percentage; calculate the member's coinsurance and the plan's covered amount, invoking the maximum logic to cap the plan's payment.

h. **Stop** the tier loop early if the remaining covered amount reaches zero(the whole charge has been allocated).

Stage F — New York State Surcharge (when applicable) Runs only when the provider's surcharge code begins with `NY` ** and** the member is not surcharge-excluded.

1. **Determine the surcharge election indicator** — from the operator-location table if present, otherwise from the member's class/SOC election indicators.

- If the SOC accounting type is "M" or "I" (and not Federal), force election
to "E" (Elect).

- If the election is "Federal" (`F`) or blank surcharge amount is zero; exit.

2. **Classify the service** as Clinical-Lab, X-Ray, or Other:

- Procedure codes 80000–89999 Clinical Lab.

- Procedure codes 70000–79999 X-Ray.

- Otherwise, classify by the type-of-service code; bypass service types(referrals, COB, drugs, etc.) are excluded.

3. **Determine whether to apply the surcharge** (`ApplySSCInd`) based on the plan schedule's 6th character, the patient relationship, and whether this is an "all-9s" surcharge-only line. Cigna-secondary situations and certain service types turn the surcharge off.

4. **GME rate** — for `NY1` non-elect providers, look up the GME region by ZIP and retrieve the GME surcharge percentage from the state surcharge table.

5. **Standard surcharge rate** — look up the percentage by state code and election indicator.

6. **Calculate the surcharge amount**:

- For `NY1` non-elect inpatient/unknown providers, add a GME component(contracted rate × GME rate).
- Add the standard component (contracted rate × surcharge percentage).
- Round to 2 decimals.

7. The surcharge amount and indicator flow into the provider's response(`ny_ssc_amt`, `ny_ssc_ind`).

Stage G — Assemble the response

22. Build the member-level response (employee ID, patient number, request date, provider counts, overall return code).

23. Add each **success provider** with its calculated cost-share totals and a per-tier ("order") breakdown.
24. Add each **error provider** with empty cost-share placeholders and its original code.
25. Set the overall **return code**:

- `200` if all providers succeeded,

- `201` if some succeeded and some errored (partial),

- `202` if all providers errored/bypassed.

- 4. Business Rules (numbered)

Request acceptance

1. A transaction with overall status other than 200 is rejected as a bad request.

2. A non-numeric member message code is rejected as a bad request.

3. A request with zero providers returns "all providers bypassed" (202).

4. Only member codes `0200` (success) or `0201` (partial) are processed; anything else is returned as an error.

5. Only providers with code `0200` are calculated; all others pass through as errored providers.

6. If no provider is calculable, the whole request is returned as an error.

Field validation (member)

7. Employee SSN must be present and a positive number (`6001` missing, `6002` invalid).

8. Patient number must be greater than zero (`6003`).

9. Request date must be present and a positive number (`6005` missing, `6006` invalid).

10. SOC number must be greater than zero (`6007`).

11. SOC class must be present and positive (`6009` missing, `6010` invalid).

12. SOC effective date must be present and positive (`6011` missing, `6012`invalid).

Field validation (per provider)

13. Priced (contracted) amount must be greater than zero (`6019`).

14. Select group must be greater than zero (`6025`).

15. Calc group must be greater than zero (`6027`).

16. A provider with a reason-denied code but no calc group is bypassed(`6024`).

17. A provider TLC code shorter than 3 characters is rejected (`6043`).

Reference-data rules

18. The plan lookup key is SOC(5) + class effective date + line-of-business +SOC class(4).

19. Coverage Detail (GCCVGDCM) is mandatory; absence is fatal (`6029`).

20. Eligibility data is mandatory only for `PCLM` claim engine (`6002`); other sources tolerate its absence.

21. The eligibility/salary record used is the first one effective on or before the request date.

Benefit definition / tiering

22. A Coverage Definition holds up to 5 benefit tiers, processed in order.

23. Processing of tiers stops at the first tier whose coverage code is 0.

24. A tier whose Medicare/accident-injury indicators do not match the member's situation is skipped.

25. A member is treated as Medicare-eligible when they have reached age 65 by the date of service.

26. A TLC 3rd character of "J" identifies an accident/injury service.

Deductible rules

27. Deductible groups are linked by **connectors** that combine them:

    - `OR / OR+ / OR-` take the **lower** remaining amount of the two groups.

    - `AND / AN+ / AN-` take the **higher** remaining amount.

    - `AN1` "apply and continue" — accumulate this group's amount and move to the next, applying against a coinsurance group.

    - `AFT` "after" — this group applies only after a connected limit is met.

    - `(O)` or blank end of the chain.

28. Deductible **types** determine how the limit is computed:

    - `A` = fixed dollar amount.

    - `O` = number of occurrences (visits).

    - `T` = occurrence-based; type `T` on the *first* group is invalid (`6008`).

    - `%` = percentage of the covered amount.

    - `S` = salary-based (uses the member's salary and salary factor).

    - `B` / `C` = salary-percentage of the charged amount, capped by a maximum limit.

    - `W` = waived (when accident-injury waiver applies and the service is an injury).

    - Any other type is invalid (`6009`).

29. A deductible/copay can "apply to OOP" (out-of-pocket) — when it does, its amount is also added into the coinsurance/OOP accumulators.

30. The deductible charged to the member is capped at the covered amount and at the remaining percentage area.

31. Copays (`DedCopayInd = "C"`) are tracked separately from deductibles but follow the same group/connector mechanics.

Coinsurance rules

32. If the Coverage Definition's "percent-pay indicator" is `P`, coinsurance is a flat percentage with no coinsurance group.

33. Otherwise the engine follows the chain of coinsurance groups, each of which can have up to 4 **cutoff levels** with their own limit and payable percentage.

34. The starting cutoff level is the first level whose limit exceeds the member's prior utilization.

35. When multiple connected coinsurance groups exist, the engine applies the one with the **least remaining amount**.

36. A cutoff amount of `9999999.99` means "pay at 100%" (no member coinsurance for that level).

37. Salary-based cutoffs (`PctSalInd = "S"`) are converted to dollar amounts using the member's salary, bounded by the group's min/max cutoffs.

38. The plan's covered expense for coinsurance is the **smaller** of the OOP-based amount and the maximum-based amount.

39. Member coinsurance = covered expense × (1 payable percentage); the plan pays covered expense × payable percentage.

Maximum rules

40. Maximums can be **amount-based** (`A`) or **occurrence-based** (`O`).

41. `MaxUnlimitedInd = "Y"` or a zero limit means **unlimited** (modeled as a very large number).

42. If utilization already meets/exceeds the limit, the maximum is "met": the plan pays nothing for that tier, the covered amount becomes not-covered, and the maximum's reason-denied code is set.

43. Maximum groups are chained by connectors (`(B)` = bypass/continue, `(O)` = end) and accumulation periods (calendar year, policy year, lifetime, rolling year, per-admission, per-service-date, etc.).

44. The plan's payment is reduced to fit within the remaining maximum; if the remaining maximum is zero, a not-covered reason is produced.

Accumulator matching

45. A prior accumulator applies to the current calculation only when its ID matches the benefit group **and** the request date falls within the accumulator's start/end date range.

46. As deductible, copay, and coinsurance are calculated, the engine **adds the newly used amounts back into the working accumulators** so connected groups see up-to-date utilization.

Output rules

47. All monetary outputs are rounded to 2 decimal places.

48. Customer cost share = deductible + copay + coinsurance + not-covered.

49. The plan (Cigna) share equals the total maximum-covered amount.

50. Overall return code is 200 (all success), 201 (partial), or 202 (all errored).

- 5.Decision Matrix

5.1 Request-level routing

Condition	Outcome
Overall status 200	Bad-request error response
Member code not numeric	Bad-request error response
No providers	Return 202 (all bypassed)
Member code not `0200`/`0201`	Error response with that code
No success providers	Error response
Some providers good, some bad	Calculate good, pass through bad 201

5.2 Member field validation error code

Field	Missing	Invalid
SSN	6001	6002
Patient number	6003	—
Request date	6005	6006
SOC number	6007	—
SOC class	6009	6010
SOC effective date	6011	6012

5.3 Provider validation error code

Condition	Code
Priced amount 0	6019
Select group 0	6025
Calc group 0	6027
Reason-denied present but no calc group	6024
TLC code < 3 chars	6043
Calc group definition not found	6028
Coverage def/detail lookup failed	6029

5.4 Deductible type limit basis

Type	Limit is based on
A	Fixed dollar amount
O / T	Number of occurrences (× occurs factor)
%	Percentage of covered amount
S	Member salary (× salary factor)
B / C	Salary % × charged amount, capped by max limit
W	Waived (injury + waiver indicator)

5.5 Coverage tier Medicare / injury skip

Tier indicator	Member condition	Result
AccInj = Y	TLC J (not injury)	Skip tier
AccInj = N	TLC = J (injury)	Skip tier
MedElig = Y	Not Medicare-eligible	Skip tier
MedElig = N	Medicare-eligible	Skip tier

5.6 NY surcharge applicability

Condition	Result
Surcharge code not `NY*`	No surcharge
Member surcharge-excluded (`ssc_excl_ind = Y`)	No surcharge
Election = Federal (`F`) or blank	Surcharge = 0
`NY1` + Non-elect + ApplySSC = Y	GME rate + standard rate
ApplySSC = N	Rates set to 0
Otherwise (NY, elect/applies)	Standard surcharge rate

5.7 Overall return code

Providers	Return code
All succeeded	200
Mixed success/error	201
All errored / bypassed	202

- 6. Exception & Error Handling

  - **Two error vocabularies are used:**

  - **Member-level codes** (`MemMap`) describe transaction- or member-wide failures (e.g. `6002` invalid SSN, `6029` coverage lookup failed).

  - **Provider-level codes** (`PrvMap`) describe failures while calculating an individual provider's benefits (e.g. `6008` deductible type T on first grou p, `6043` error processing ded/copay/coi/max).

  - **Fail-safe per provider:** a failure while calculating one provider does **not** abort the whole transaction. That provider receives its error code a nd the others still calculate; the transaction returns 201 (partial) when appropriate.

  - **Database/reference lookup failures:**

  - Missing Coverage Detail (GCCVGDCM) member error `6029` (fatal).

  - Missing eligibility for PCLM member error `6002` (fatal).

  - Missing surcharge / ZIP / state data the surcharge is set to zero and a warning is logged (`6091` DB call failed); calculation continues.

  - **Maximum met** is a *business* outcome, not a technical error: the tier is closed and a "reason not covered" code is populated, but the response is still successful (`0200`).

  - **Date conversion failures** during accumulator matching produce internal errors (codes in the `6049`–`6060` range) tied to the original mainframe date routines.

  - **Unhandled internal errors** while processing a provider fall back to provider code `6043` with a generic "error processing ded/copay/coi/max" message.

  - **Infrastructure errors** (config load, logger, DynamoDB client, database connection) are fatal at startup — the service will not start.

  - **HTTP layer:** request decode failures 400; calculation failures 500; otherwise 200 with the calculated body.

- 7. Outputs

The service returns a structured response containing:

Member section

- Employee ID, patient number, request date
- Number of requested providers and benefit-calc groups
- Overall **return code** (200 / 201 / 202 / error code)

Per-provider section (success providers)

- Provider ID, PPO ID, provider association, network indicator, priced amount

- **Customer cost share** (member liability)

- **Plan (Cigna) share**

- **Total deductible**, **total copay**, **total coinsurance**, **total OOP**

- **Total not-covered amount**

- **NY surcharge amount and indicator**

- Per-provider return code

Per-tier ("order") breakdown (up to 5 per provider)

- Deductible amount / limit / remaining / utilized / type
- Copay amount / limit
- Coinsurance amount / limit / percentage / utilization / remaining / type
- Maximum group / limit / amount / utilized / remaining / type
- Not-covered amount and reason code
- Whether deductible/copay apply toward out-of-pocket
- Member out-of-pocket cost share for the tier

Per-provider section (error/bypassed providers) — same envelope with empty cost-share placeholders and the original message code.

Metadata / outcome — status, message, code, optional additional details.

> **Note on response shapes:** The internal calculation result

> (`TransResponse`) is the structure returned by the Lambda handler. A

> "digital" response shape (`DigitalRes`, seen in `response.json` with

> `member` / `providers` / `orders` / `metadata`) defines the external,

> consumer-facing field names (e.g. `cust_cost_share`, `cigna_share_amount`,

> `ny_ssc_amt`). Both describe the same calculated values.

- 8. Key Assumptions

1. **Service date = request date.** The service currently treats the request date as both the start and "thru" date of service; multi-day spanned-date logic exists in the code but is exercised as a single day.

2. **Prior accumulators are supplied by the caller.** The service does not read the member's historical usage from a database; it relies on the `accums` arrays in the request and augments them in-memory during calculation.

3. **Line of business (LOB)** used in the plan key is a fixed constant in the code.

4. **`PCLM` is the strict claim engine.** Only `PCLM` requires eligibility data; other sources (e.g. SAR) bypass some lookups.

5. **NY is the only state surcharge implemented.** Surcharge logic activates only for codes beginning with `NY`; other state codes are ignored.

6. **Mainframe parity is intentional.** Many constants (e.g. `9999999.99` as "unlimited", the `+0.0055` rounding nudge in occurs-factor math, the `CalcNumDays = 1` placeholders) replicate the original COBOL program and are preserved deliberately rather than re-derived.

7. **Up to 5 tiers and bounded chains.** A Coverage Definition supports at most 5 benefit tiers; connected deductible/coinsurance/maximum chains are bounded (e.g. 18–30 iterations) to prevent infinite loops, matching mainframe table sizes.

8. **Rounding** is applied to 2 decimal places at each monetary output; intermediate values may carry more precision.

9. **Parallel processing** of providers and reference-data lookups is a performance optimization and does not change the business result; each pr
ovider is calculated independently.

10. **Medicare eligibility** is inferred purely from age (65 by date of service) and plan indicators, not from external Medicare data.

CDC and Initial Load: MVP1

- Create the process to load the GQDN.CXT_RNC_XWALK table into the Dynamo table CXT-RNC-XWALK.
  - Currently production has 1619 rows
- Write the process to load the GM.GMTBPPOR table everyday into new Dynamo DB. QLIK replication
  - Process must load all the records everyday.
  - New Dynamo DB name would be PPO-DEFINATION.
  - Key would be Product and sort KEY is PPOID.
  - Currently production has 4798 rows
- Write the process to load the GM.TBALLPPO table everyday into new Dynamo DB.
  - Process must load all the records everyday.
  - New Dynamo DB name would be ALLPPO-NETWORKS.
  - Key would be Product and sort KEY is Hierarchy.
  - Currently production has 2301rows

- Write the process to load the GM.TBZIPTOP table everyday into new Dynamo DB.
  - Process must load all the records everyday.
  - New Dynamo DB name would be TBD.
  - Key would be Product and sort KEY is Hierarchy.
  - Currently production has 4470421 rows.

- Write the process to load the GM.GMTBCSSM table everyday into new Dynamo DB QLIK replication
  - Process must load all the records everyday.
  - New Dynamo DB name would be Contract-methodology
  - Key would be Product and sort KEY is Hierarchy.
  - Currently production has 963 rows.
- Write the process to load the GQDN.RNC_SANP_XWALK table everyday into new Dynamo DB. QLIK replication
  - Process must load all the records everyday.
  - New Dynamo DB name would be SANP-RNC-XWALK.
  - Key would be RNC-TYP-CD sort KEY is RNC-CD-VALUE.
  - Currently production has 296 rows
- Write the process to load the GQDN.CMN_CONFGRTN2 table everyday into new Dynamo DB. QLIK replication (Kishore will confirm)
  - Currently production has 1114 rows